6.2. Lock-based concurrent data structures

6.2.1. A thread-safe stack using locks

6.2.2. A thread-safe queue using locks and condition variables

6.2.3. A thread-safe queue using fine-grained locks and condition variables

6.3. Designing more complex lock-based data structures

6.3.1. Writing a thread-safe lookup table using locks

6.3.2. Writing a thread-safe list using locks

6.4. Summary

Chapter 7. Designing lock-free concurrent data structures

7.1. Definitions and consequences

7.1.1. Types of nonblocking data structures

7.1.2. Lock-free data structures

7.1.3. Wait-free data structures

7.1.4. The pros and cons of lock-free data structures

7.2. Examples of lock-free data structures

7.2.1. Writing a thread-safe stack without locks

7.2.2. Stopping those pesky leaks: managing memory in lock-free data structures

7.2.3. Detecting nodes that can’t be reclaimed using hazard pointers

7.2.4. Detecting nodes in use with reference counting

7.2.5. Applying the memory model to the lock-free stack

7.2.6. Writing a thread-safe queue without locks

7.3. Guidelines for writing lock-free data structures